

emlearn hands-on

Lab exercises
Human Activity Recognition

TinyML Days Aarhus 2025
Jon Nordby jononor@gmail.com

Materials

Presentation

https://docs.google.com/presentation/d/1ldizH74QPTS3Ms1ejdJ_YFvH5yn2yaKxZuTMUZrg7a8/edit?slide=id.g343d3721787_0_23#slide=id.g343d3721787_0_23

emlearn documentation

<https://emlearn-micropython.readthedocs.io/en/latest/>

<https://emlearn.readthedocs.io/en/latest/>

MicroPython documentation

<https://docs.micropython.org/en/latest/>

Software preparations

Make sure to have **Python 3.10+** installed and on PATH

Make sure to have **git** installed and on PATH

Download materials for workshop

git clone <https://github.com/emlearn/emlearn-micropython/>

Go to emlearn-micropython/examples/har_trees

Create virtualenv

```
python -m venv emlearn_venv
```

Activate virtualenv

Linux/Mac/WSL (bash)

Windows PowerShell

Windows cmd.exe

```
source emlearn_venv/bin/activate
```

```
emlearn_venv/Scripts/Activate.ps1
```

```
venv\Scripts\activate
```

**! On Windows,
may need this before activating**

```
Set-ExecutionPolicy -ExecutionPolicy
```

```
Unrestricted -Scope CurrentUser
```

5. Install har_trees requirements

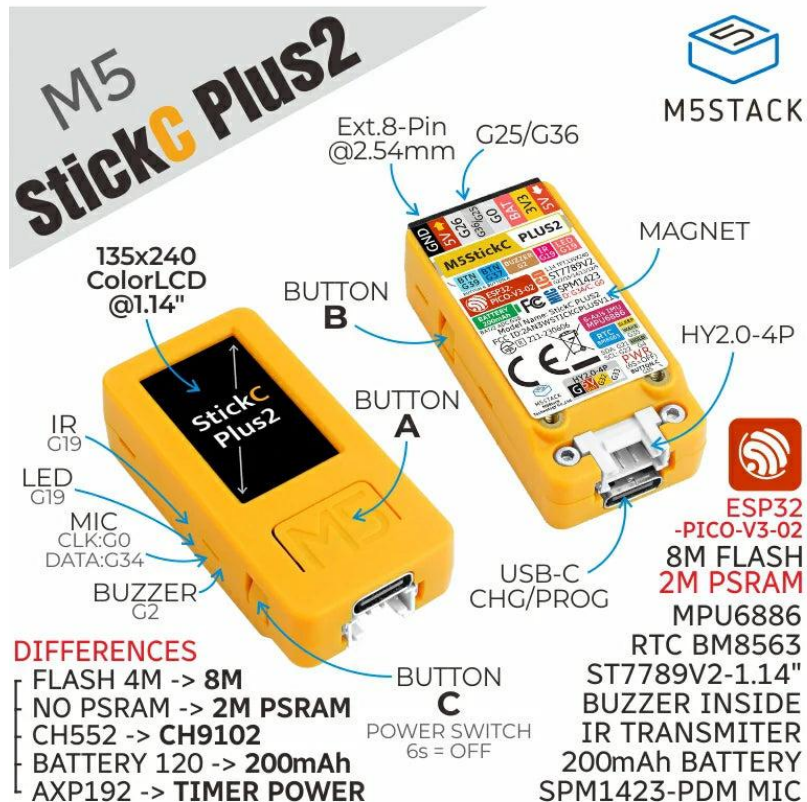
```
pip install -r requirements.txt
```

Hardware Preparations

M5Stick C PLUS 2 - from M5Stack

Pre-flashed with MicroPython 1.25
(from <https://micropython.org>)

! May need USB Serial driver (CH9102) - Link on website
<https://docs.m5stack.com/en/core/M5StickC%20PLUS2>



Exercise 1 - Train model using standard dataset

(in the folder har_trees, with emlearn_venv active)

Get the latest version of files

```
git pull origin master
```

Download dataset

```
python -m leaf_clustering.data.har.uci
```

Run training process

```
python har_train.py
```

Results written to ./uci_har.results.parquet

Results

	n_estimators	min_samples_leaf	mean_train_f1_micro	mean_test_f1_micro
0	10	1	0.998905	0.809926
1	10	4	0.992164	0.811002
2	10	16	0.964593	0.812353
3	10	64	0.902257	0.811473
4	10	256	0.863699	0.819042

Exercise 2 - Run model on device

Check connection

```
mpremote exec "import sys; print(sys.implementation)"
```

```
name='micropython', version=(1, 25, 0, ""),  
_machine='Generic ESP32 module with ESP32',  
_mpy=11014, _build='ESP32_GENERIC')
```

Install packages on device

```
mpremote mip install https://emlearn.github.io/emlearn-micropython/builds/master/xtensawin\_6.3/emlearn\_trees.mpy  
mpremote mip install github:jonnor/micropython-npyfile  
mpremote mip install github:jonnor/micropython-zipfile  
mpremote mip install github:jonnor/micropython-mpu6886  
mpremote mip install github:peterhinch/micropython-nano-gui/drivers/st7789  
mpremote mip install "github:peterhinch/micropython-nano-gui"  
mpremote mip install "github:peterhinch/micropython-async/v3/primitives"
```

Copy local project files (NOTE: the colon at end is critical)

```
mpremote cp uci_har.trees.csv uci_har.meta.json uci_har.testdata.npz timebased.py windower.py recorder.py color_setup.py :
```

Run evaluation on test set

```
mpremote run har_run.py
```

Run with live predictions

```
mpremote run har_live.py
```

Exercise 3a: Record own data

1. Set the time

```
mpremote rtc --set
```

2. Start recording program

```
mpremote run har_record.py
```

3. Record 20 seconds of data per class

Select class: Double-click “M5” button

Start/stop recording: Long-press “M5” button

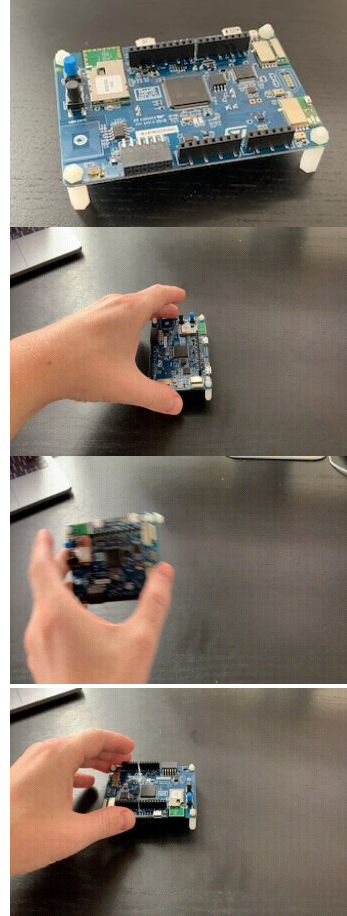
(if needed) Delete recordings from device

```
mpremote exec "from recorder import Recorder; r = Recorder(100, 1.0, directory='har_record'); r.delete()"
```

Motions to record

- Idle
- Updown
- Wave
- Snake

<https://docs.edgeimpulse.com/nordic/pre-built-datasets/continuous-gestures>



Exercise 3b: Train and deploy custom model

Make dataset directory

```
mkdir ./data/raw/cgestures-2
```

Copy files

```
mpremote cp -r ./har_record ./data/raw/cgestures-2
```

Label* data

```
python har_quicklabel.py --data-dir data/raw/cgestures-2/har_record/ --dataset cgestures-2
```

Train model

```
python har_train.py --dataset cgestures-2 --config data/configurations/cgestures.yaml
```

Open `har_live.py` in **text editor** - change to `DATASET='cgestures-2'`

Copy the new model to device

```
mpremote cp cgestures-2.trees.csv cgestures-2.meta.json :
```

Run

```
mpremote run har_live.py
```


Exercise 3c: Test your model

1. Do the different gestures *in the same way* as during training

The device should be able to recognize it (at least most of the time)

2. Try to trick the model

How can you position or move the device
to make the model output incorrect predictions?

Why do you think this is happening? **Discuss**

What could we do to remedy this?

Exercise 4: Improve your model

1. Write down the different orientation/motions you have identified **and which class you want it to be**

This will form your data gathering plan

2. Record a new dataset “cgestures-3” - with all the variations above
(refer back to exercise 3a)
3. Train and deploy the new model
(refer back to exercise 3b)
4. Test the updated model

Exercise 5: Make a model for a custom set of activities

Using what you have learned in the previous exercises

Ideas:

- Other kinds of gestures
- Physical exercises / gym activities
- Everyday activities at home/office

Things to think about

- That there is practical and way to get data
- Where device should be placed.
In hand / On wrist / in pocket / other?
- Should be continuous/repeating motion

Exercise 6: Do something different with model predictions

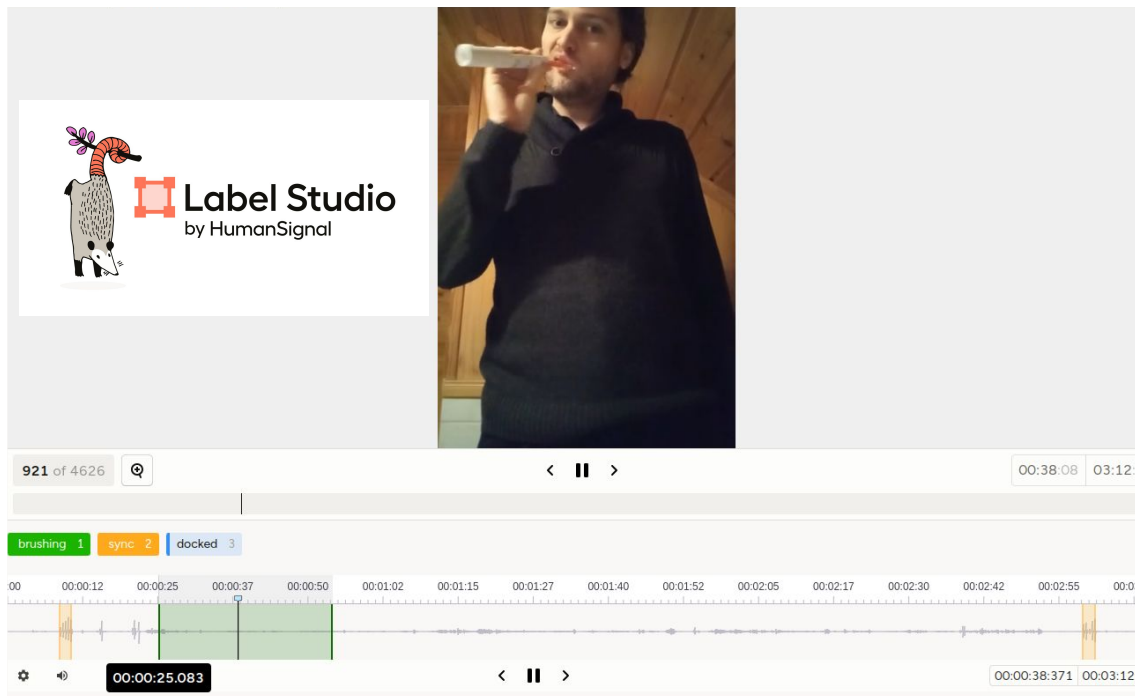
So far have only counted up the durations and show on screen

Input/output possibilities:

- Indicate using the on-device LED
- Different UI on screen/buttons
- Use the on-device buzzer for sound
- Send to a server over WiFi. HTTP/MQTT
- Send to PC or phone over Bluetooth Low Energy

Improvements

Better labeling



Video recorded on phone
used as reference for labeling

start/end marked by tapping device
- used to sync video/labels from
phone with sensor data

Use Label Studio for labeling

! now supports time-series together
with video

https://labelstud.io/templates/timeseries_audio_video

Data collection tools:

https://github.com/emlearn/emlearn-micropython/tree/master/examples/har_trees

Better feature extraction

Currently, only have basic time-domain features

- Gravity separation using IIR filter
- Spectral data, using FFT
- Sensor fusion to incorporate gyro (6-axis IMU)
- Wavelet transform
- Spectrogram using FFT (for deep learning models)

Bonus

USB passthrough permissions in WSL

Need to install USB forwarding

<https://devblogs.microsoft.com/commandline/connecting-usb-devices-to-wsl/>

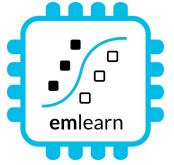
Optional: <https://gitlab.com/alelec/wsl-usb-gui>

Add WSL user to dialout group

```
sudo adduser USER dialout
```

```
newgrp dialout
```

Installing MicroPython



Download prebuilt firmware

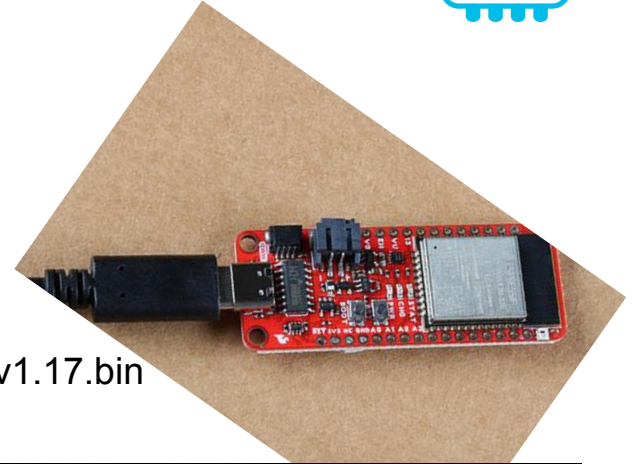
<https://micropython.org/download/?port=esp32>

Flash firmware to device

```
pip install esptool
```

```
esptool.py --chip esp32 --port ... erase_flash
```

```
esptool.py --chip esp32 --port ... write_flash -z 0 micropython-v1.17.bin
```



Connect to device

```
pip install mpremote
```

```
mpremote repl
```

```
MicroPython v1.8.3-24-g095e43a on 2016-08-16; ESP module
Type "help()" for more information.
>>> print('Hello world!')
Hello world!
>>> █
```

IDE (optional): Viper IDE, Thonny, VS Code, et.c.

Using Thonny

Install Thonny with CPython 3.10 bundled

Create folder `emlearn_venv`

TODO: screenshot how to create virtualenv in Thonny

TODO: screenshot how to install requirements.txt in Thonny

TODO: screenshot how to open system shell in Thonny